Copilot

Sure, here are some examples of Java code that correspond to each type of time complexity:

1. **Constant Time: O(1) -** An example of constant time complexity is accessing an element in an array by its index.

```
int[] arr = {1, 2, 3, 4, 5};
int element = arr[3]; // This operation is O(1)
```

1. **Linear Time: O(N) -** An example of linear time complexity is traversing through an array or list.

```
int[] arr = {1, 2, 3, 4, 5};
for(int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]); // This operation is O(N)
}
```

1. **Logarithmic Time: O(log N) -** An example of logarithmic time complexity is the binary search algorithm.

```
public int binarySearch(int arr[], int l, int r, int x) {
    if (r >= l) {
        int mid = l + (r - l) / 2;
        if (arr[mid] == x)
            return mid;
        if (arr[mid] > x)
            return binarySearch(arr, l, mid - 1, x);
        return binarySearch(arr, mid + 1, r, x);
    }
    return -1; // This operation is O(log N)
}
```

1. **Linear Logarithmic Time: O(N log N) -** An example of linear logarithmic time complexity is the merge sort algorithm.

```
void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r); // This operation is O(N log N)
    }
}
```

1. **Quadratic Time: O(N^2) -** An example of quadratic time complexity is the bubble sort algorithm.

```
void bubbleSort(int arr[]) {
    int n = arr.length;
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            } // This operation is O(N^2)
}
```

1. **Cubic Time: O(N^3) -** An example of cubic time complexity is a naive matrix multiplication algorithm.

```
void multiply(int A[][], int B[][], int C[][]) {
    int N = A.length;
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            C[i][j] = 0;
```

```
            for (int k = 0; k < N; k++)
                C[i][j] += A[i][k] * B[k][j]; // This operation is O(N^3)
        }
    }
}
```

1. **Exponential Time: O(2^N) -** An example of exponential time complexity is the recursive calculation of Fibonacci numbers.

```
int fibonacci(int n) {
    if (n <= 1)
        return n;
    return fibonacci(n - 1) + fibonacci(n - 2); // This operation is O(2^N)
}
```

1. **Factorial Time: O(N!) -** An example of factorial time complexity is generating all permutations of a string.

```
void permute(String str, int l, int r) {
    if (l == r)
        System.out.println(str);
    else {
        for (int i = l; i <= r; i++) {
            str = swap(str, l, i);
            permute(str, l + 1, r);
            str = swap(str, l, i); // Backtrack to restore the original string
        } // This operation is O(N!)
    }
}
```

Remember, these are just examples and the actual time complexity can vary based on the specific details and requirements of your code. Also, these examples do not include the code for helper methods like `merge` for merge sort and `swap` for string permutation. You would need to implement those as well. I hope this helps! Let me know if you have any other questions. 😊